第十四届“恩智浦”杯全国大学生

智能汽车竞赛

技 术 报 告



学　　校：黑龙江工程学院

队伍名称：睿龙七队

参赛队员：赵智飞 马洪岩 李怡鑫 李柏春 侯聪

带队教师： 崔宏耀 齐益强

关于技术报告和研究论文使用授权的说明

本人完全了解第 14 届“恩智浦”杯全国大学生智能汽车竞赛关保留、使用技术报告和研究论文的规定，即：参赛作品著作权归参赛者本人，比赛组委会和恩智浦半导体公司可以在相关主页上收录并公开参赛作品的设计方案、技术报告以及参赛模型车的视频、图像资料，并将相关内容编纂收录在组委会出版论文集中。

参赛队员签名：赵智飞 马洪岩 李怡鑫
李柏春 侯聪

带队教师签名： 崔宏耀 齐益强

日　　期： 2019 年 8 月 18 日

技术报告

1.系统整体设计

1.1 系统概述

1.2 整车布局

2.智能车机械结构调整与优化

2.1 传感器安装

3.硬件系统设计与实现

3.1 主控芯片

3.2 电源部分

3.3 单片机 3.3V 供电

3.4 路由器/雷达 5V 供电

3.5 雷达转 usb

3.6 电源板原理图

4.软件系统设计与实现

4.1 整体 ROS 通信

4.2 ROS 开发工具与实现

4.3 使用导航

4.4 电脑与底盘主控芯片通讯协议编写

4.5 amcl 参数的设置

4.6 move_base 主要参数的调整

5.使用导航算法

1.系统整体设计

1.1 系统概述

第十四届创意组室外组比赛规则要求车模根据已构建好的地图自主导航，避开锥桶障碍物，从赛道起点跑到终点，记录比赛完成时间，根据完成时间进行比赛成绩排名。所以任务集中在整体路径规划，局部路径规划，处理异常行为。

阅读比赛要求以及查阅相关书籍,可以得知本次室外光电组智能车是一个自主移动机器人（采取 ROS 系统），即可以从当前位置自动移动到目标位置。借助钢铁侠公司官方提供的车模和提供的例程以及 ROS 生态下强大的软件包，可以完成比赛任务。

机构各模块功能简述如下：

激光雷达

选用镭神 LS01G 三角法激光雷达，测量角度为 360°，最大测量范围⩾8 米，测量频率 3600~4000Hz，测量精度 1m 以内为毫米级，1m 以上为实际距离的 1%，角度分别率为 1°，

扫描频率 3~11Hz。

姿态传感器

使用 ART-IMU-02A 姿态传感器，姿态角测量范围姿态角：测量范围(pitch/roll):±90/±180 度,动态精度:0.5 度,分辨率:0.1 度;航向角(yaw):±180 度,动态精度:2(RMS)

分辨率：0.1 度；陀螺仪：测量范围(pitch/roll/yaw):±1000 度/s，零偏稳定性：50 度/h

非

线性度：0.2%FS，加速度计：三轴测量范围:±2g，零偏稳定性:5mg，非线性度:0.5%FS；

磁力计：三轴测量范围:±12Guass，分辨率:0.003Guass，分线性度:0.1%FS；气压计：

高

度分辨率:1cm，测量范围:10~1200mbar。

直流无刷电机

使用 ART-M1-K2150 直流无刷电机，kv 值为 2150，功率为 2400w，最高电压 19V，最

高

转速 45000rpm。

舵机

使用 ART-SG995 舵机，工作频率为 1520μs/330Hz，工作电压为 4.8V~6.0V。

1.2 整车布局：钢铁侠官方提供整体布局，我们仅在此基础上适当改装调整。

1.3 需要解决问题

（1）Linux 系统的熟悉

（2）ROS 系统的掌握

（3）各种传感器数据的融合

（4）定位的准确，减少漂移

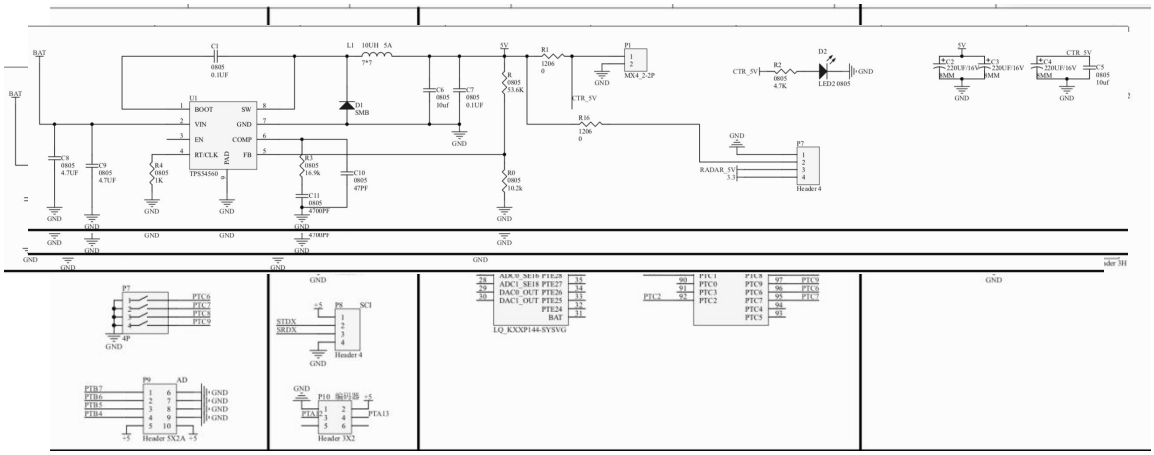
2.智能车机械结构调整与优化

2.1 传感器安装：IMU 安置在车辆前方

增加 512 线编码器件

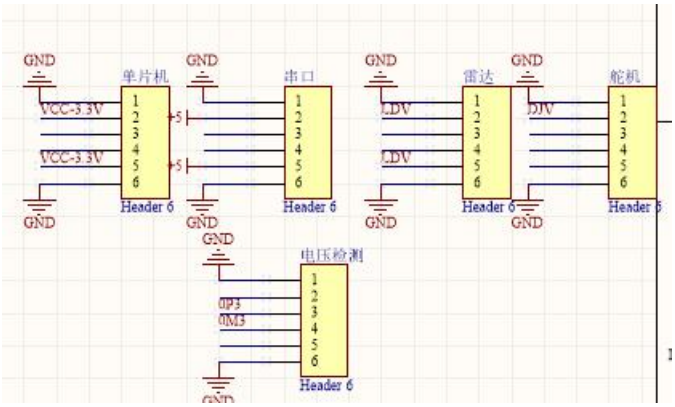
3.硬件系统设计与实现

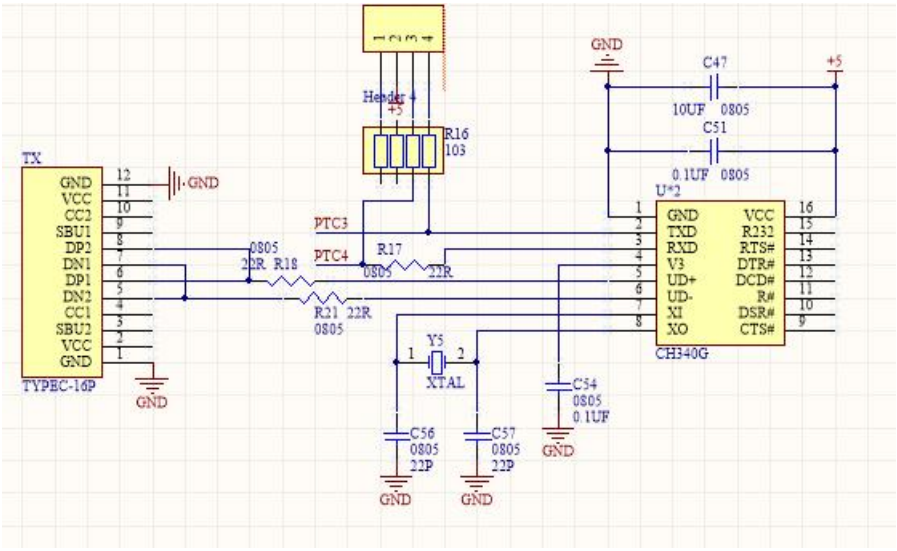
3.1 主控芯片

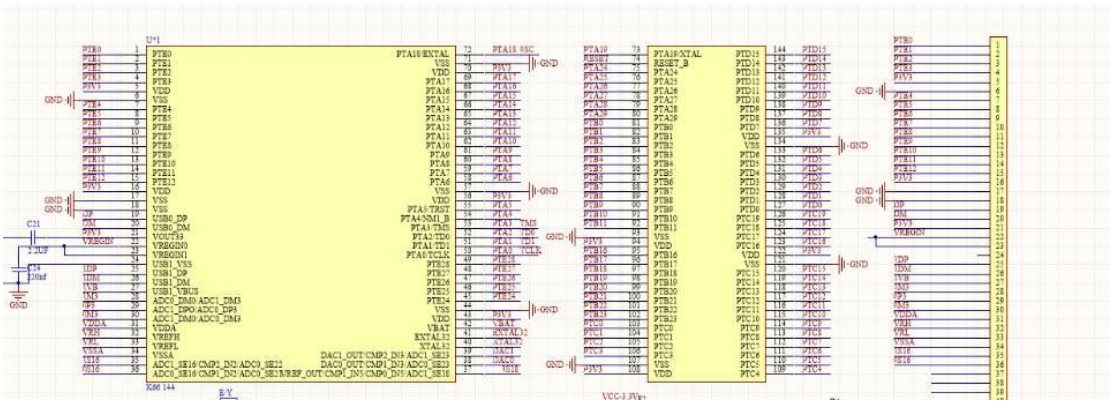


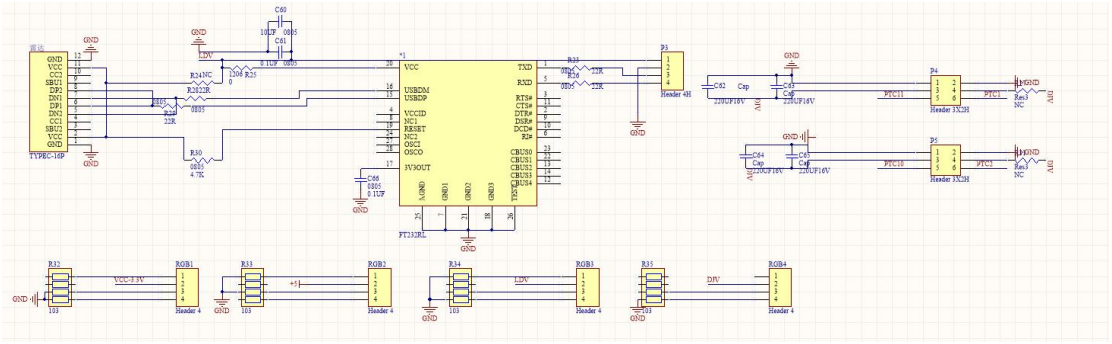
采用 NCP 提供的 K66 芯片，主频为 200mhz 使用了两路 pwm 输出，其中一路给电调提供信号，另一路为舵机提供信号。与主机之间通过 CH340G 将串口信号转化为 USB 信号，采用 USB-B 接口。

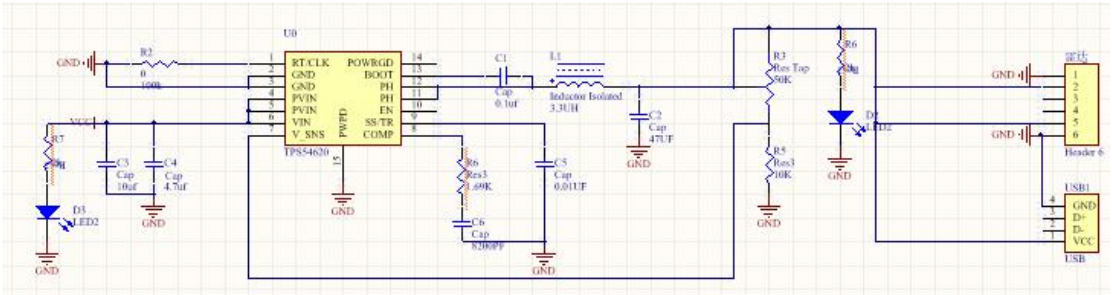
剩余电路图如下




TX
TYPEC-16P
CH340G
XTAL
Header 4
R16
103
C47
10UF 0805
C51
0.1UF 0805
C54
0805
0.1UF
C56
0805
22P
C57
0805
22P
R17
22R
R18
22R
R21 22R
0805
PTC3
PTC4
GND
+5






U0
TPS54620
L1
Inductor Isolated
3.3UH
C1
C2
47UF
R3
50K
R5
10K
R6
D2
LED2
D3
LED2
USB1
USB

4.软件系统设计与实现

4.1 整体 ROS 通信

Topic 话题是节点用来传输数据的总线。通过话题进行消息路由不需要节点之间直接连接，这 就意味着发布者和订阅者之间不需要知道彼此的存在。同一个话题可以有多个订阅者，也可 以有多个发布者。话题默认采用 TCP/IP 长连接的方式传输。

话题之间的通信是通过在节点之间发送 ROS 消息实现的。对于发布器和订阅器之间的通信，发布器和订阅器之间必须发送和接收相同类型的消息。这意味着话题的类型是由发布在它上面的消息类型决定的。

Servicev 当需要直接与节点通信并获得应答时，将无法通过话题实现，而需要使用服务。

服务(services)是节点之间通讯的另一种方式。服务允许节点发送请求(request) 并获得一个响应(response)。

服务调用非常适用于那些只需要偶尔去做并且会在有限时间里完成的事情。

Action 动作，此机制非常适合作为时间不确定目标导向型的操作的接口。

4.2 ROS 开发工具与实现：

使用 roboware studio 方便查看编写调试。

4.3 使用导航

机器人所能做的最基本的一件事就是在真实环境中动来动去。为了更有效地移动，机器人需要知道它自己在哪里，需要到哪里去。为了做到这件事，我们需要给机器人一个环境地图，

一个起始位置，以及一个目标位置。

使用 ROS 的 amcl 包在地图中定位机器人。amcl 节点实现了一系列概率定位算法，总称为自适应蒙特卡洛定位算法（Adaptive Monte Carlo Localization)

机器人的定位，也叫作它的 pose（位姿），是由 map coordinate frame(地图坐标系，有时候也叫做世界坐标系）中的一个位置和方向表示的。amcl 维护的一系列位姿，用来表示机器人可能的位置。这些 candidate pose(候选位姿）中的每一个都有一个与之关联的概率；概率越大表示机器人处于这一位姿的可能性越大。机器人到处移动过程中，程序根据地图和某个给定位姿算出一个理想的传感器数据，并与实际的传感器数据相比较。对于每个候选位姿，如果比较的结果是一致的，那么这个位姿的可信度就会提高，反之如果不匹配，则可信度下降。随着时间的推移，可信度低的位姿（也就意味着机器人不太可能处于这些位姿）就会被去除，可信度低的位姿（也就意味着机器人不太可能处于这些位姿）就会被去除，可信度高的被留下来。随着机器人的移动，候选位姿也随之移动，一直跟随着估计出来的机器人里程计数据。

一个完成定位的机器人，我们尝试着让它动一下。这个导航系统被前辈称作 navstack 也就是导航软件包。

导航软件包的工作原理

<1.>navigation goal(导航目的地）被发送到导航软件包中，为了做到这一点，我们进行一个（action）并传入一个 MoveBaseGoal 类型的参数表示目的地，这个参数是某个坐标系（通常是 map 坐标系）中的位姿（位置和方向）。

<2.>导航软件包中的组件 global planner（本地规划器）使用路径规划算法和地图，规划一条从当前位置到目标位置的最短路径。

<3.>这个路径被传入 local planner（本地规划器）中，本地规划器会试图驱动机器人沿

着这个路径行走。本地规划器使用传感器的信息绕开挡在机器人前面却不在地图中的障碍，比如行人。如果本地规划器被卡住无法规划出路径，它会请求全局规划器规划一条新的路径，并尝试重新跟随这条路径。

<4.>当机器人接近目的位姿的时候，行为调用就结束了。

4.4 电脑与底盘主控芯片通讯协议编写

借助钢铁侠官方提供的培训我们队伍基本了解了底盘电机和电调控制的方式以及协议的处理。

首先编写之初查看 NXP 通信转接板指令协议，可以得知定义通信中的指令格式、指令类型和指令内容。转接板可以只需要根据上位机指令来输出 pwm 波来控制电调舵机。根据其中的内容提示我们可以自己设计下发通信格式，自己进行设计，但是由于比赛时间限制我们队伍暂时采用已有格式进行承袭。

编写时主要的要点有：

<1.>pwm 波输出的频率为 50HZ，本队伍使用的单片机为 K66，应该进行分频操作，以保证电机舵机的正常工作;

<2.>使用 uart 串口通信协议，波特率为 38400，8 位数据位 1 位停止位，无奇偶校验；

<3.>要根据下发的指令格式进行相应的转化操作转化为相应的舵机电机 pwm 数值。

具体程序如下所示：

主函数：

/***

****

以单片机（k66）作为直接执行机构，以下为单片机的执行和接受命令的程序

```
void main(void)
{
    DisableInterrupts;          //关闭总中断

    PLL_Init(PLL200);           //初始化 PLL 为 200M

    PIT_Init(PIT0,10);//.....PIT 定时中断

    LED_Init();//.....LED 灯初始化

    ADC_Init(ADC_0);            //AD 采集初始化

    ADC_Init(ADC_1);

    FTM_PWM_Init(FTM0, FTM_CH0,20000, 1500);//..... 电 机 PWM 初始化//引脚 PTC2

    FTM_PWM_Init(FTM3, FTM_CH7,20000, 2080);//565  2435//..... 舵机 PWM 初始化//引脚

    FTM_AB_Init(FTM1);//.....正交解码初始化

    GPIO_Init(PTC,2, GPO,0);//.....测程序运行时间引脚

    GPIO_Init(PTB, 18, GPO,0);//.....测程序运行时间引脚
```

//GPIO_Init(PTC, 6, GPI,1);//.....调试状态

Init_LCD_PORT();//.....初始化 LCD 的端口

Lcd_Init();//.....初始化 LCD

LCD_Clear(BLACK);//.....清屏

BACK_COLOR=BLACK;//.....背景色设置为黑色

POINT_COLOR=RED;//.....画笔颜色

Show_static_info_on_lcd();//.....显示一些变量的提示信息

Show_Team_name();//.....队名信息（检查显示屏是否能正常显示）

PID_Ince_Con_Initia();//.....速度控制 PID 参数初始化

sen_var[1] = 45;

UART_Init (UART_4,38400); //..... 串口初始化,UART4,用 PTE24,E25

```
    UART_Irq_En(UART_4);//.....UART 中断开
启

    EnableInterrupts;

while(1)
    {

    tem_1=ADC_Ave(ADC_1,ADC1_DP0,ADC_12bit,100);      //电池电压采集

    fv_1=((float)tem_1)*14.38167/4096; //11.38167                    //转为电
压

    tem_2=ADC_Ave(ADC_1, ADC1_DM0,ADC_12bit,100);      //电池电压采集

    fv_2=((float)tem_2)*8.38167/4096;                    //转为电压


  if(fv_1<10.3||fv_2<7.2)              //电压低于一定值以后蜂鸣器开始报警
  {
    count++;
      if(count>100)
      {
        count=101;
        count_2++;
        if(count_2>100)
        {
          count_2=0;
          GPIO_Reverse(PTC,2);

        }


      }
  }
      //Send_Mess_to_Upper_Mec();//.....上位机
发送的指令
```

```
    Moter_Speed = (Mot_pulse *
MS_4PULSE)/M_S_PULSE/Coefficent;//.....速度计算
     //sen_var[0] = Moter_Speed*30;
    Only_Moter_motion(Want_Speed);//.....驱动电机//
参数：float speed_aim，目标速度
    //Show_Pulse_on_lcd(150,100);



#if 0    //.....山外上位机显示波形//波特
率设为:9600
          if(sen_var[1] == -45)
            sen_var[1] = 45;
          else
            sen_var[1] = -45;

          RLYD_sendware((uint8*)(sen_var), sizeof(sen_var));//波形显示
          LED_Ctrl(LED0, RVS);


#endif
   }
}
```

UART 中断中对接受数据的处理

```
/*****************************************************************************
*                       睿 龙 科 技 协 会   睿龙 7 队
*
*    函数名称：void Send_Mess_to_Upper_Mec(void)
*    功能说明：k66 单片机向上位机通信
*    参数说明：
```

```
*    函数返回：

*    修改时间：2019.07.23    22.10

*    备      注：

*************************************************************************/
void Send_Mess_to_Upper_Mec(void){


UART_Query_Str (UART_4,cpp,1);//接受方式，接受字符

  for(Upper=0;Upper<9;Upper++)//处理方式
    {
      if(HHH[Upper]==170&&HHH[Upper+6]==85)

{sump=HHH[Upper+1]+HHH[Upper+2]+HHH[Upper+3]+HHH[Upper+4];

        if((256+HHH[Upper+5]==sump)||(HHH[Upper+5]==sump))

      {
       hpp[0]=HHH[Upper];
       hpp[1]=HHH[Upper+1];
       hpp[2]=HHH[Upper+2];
       hpp[3]=HHH[Upper+3];
       hpp[4]=HHH[Upper+4];
       hpp[5]=HHH[Upper+5];
       hpp[6]=HHH[Upper+6];
       yes_ok=1;

    pwm_di=hpp[2]*256+hpp[1] - 1000;
      pwm_du=hpp[4]*256+hpp[3]+525+80;

      FTM_PWM_Duty(FTM3,FTM_CH7,pwm_du);//舵机直接输出

      }
    }

      Want_Speed = pwm_di/1000.000;//输入指令换算成目标速度


      if(yes_ok==1)
      {
        for(Upper=0;Upper<7;Upper++)
        {
```

```
            dpp[Upper]=(char)hpp[Upper];

          }
          UART_Put_Str(UART_4 ,dpp);//返回数据进行效验
          yes_ok=0;
          break;
        }

      }


}
/*****************************************************************************
*                        睿 龙 科 技 协 会   睿龙 7 队
*
*    函数名称：PID_control
*    功能说明：增量式 PID 控制系数初始化
*    参数说明：
*    函数返回：
*    修改时间：2019.07.28          21.40
*    备      注：PID 增量式
*****************************************************************************/
void PID_Ince_Con_Initia(void){

        PID_SWG.dif = 0;

        PID_SWG.l_dif = 0;

        PID_SWG.l_l_dif = 0;

        PID_SWG.PWM_Mod = 1500;

        PID_SWG.PWM_Increase = 0;

        PID_SWG.Prop = 120.0;//114.0
```

```
    PID_SWG.Inte = 2.0;//1.8

    PID_SWG.Deri = 0;
}

/*****************************************************************************
*                       睿 龙 科 技 协 会   睿龙 7 队
*
*    函数名称：PID_Incr_Control
*    功能说明：PID 控制
*    参数说明：dif：偏差
*    函数返回：
*    修改时间：2019.07.28        21.40
*    备      注：PID 增量式
*****************************************************************************/
int PID_Incr_Control(float Aim_speed, float Tru_speed){

    float dif = 0;

    //Tru_speed = 1.2;
  if(Aim_speed==0)
    {
        PID_SWG.PWM_Mod=PID_SWG.Prop*(Aim_speed - Tru_speed);

    }
    else
    {

    dif = Ampli_Coeff*(Aim_speed - Tru_speed);//.....................................本
次偏差

    PID_SWG.dif = dif;

    PID_SWG.PWM_Increase      =      (int)(PID_SWG.Prop*(PID_SWG.dif      -
```

```
PID_SWG.l_dif)
    + PID_SWG.Inte * PID_SWG.dif
    +        PID_SWG.Deri*(PID_SWG.dif        -        2*PID_SWG.l_dif        +
PID_SWG.l_l_dif));.....增量

    PID_SWG.l_l_dif = PID_SWG.l_dif;//.....偏差更新

    PID_SWG.l_dif = PID_SWG.dif;

    PID_SWG.PWM_Mod += PID_SWG.PWM_Increase;//.....增量
相加
    }
    PID_SWG.PWM_Mod = PID_SWG.PWM_Mod > Upper_PWM?
    Upper_PWM: PID_SWG.PWM_Mod;//.....限幅
    PID_SWG.PWM_Mod = PID_SWG.PWM_Mod < Floor_PWM?
    Floor_PWM: PID_SWG.PWM_Mod;

    return PID_SWG.PWM_Mod;
}
```

增量式 PID 控制

根据位置式 PID 控制公式，写出 n-1 时刻的控制量：

$$u[n-1]=K_p\left\{e[n-1]+\frac{T}{T_i}\sum_{i=0}^{n-1}e[i]+\frac{T_d}{T}\{e[n-1]-e[n-2]\}\right\}$$

设

$$\Delta u[n]=u[n]-u[n-1]$$

得到

$$\Delta u[n]=K_p\{e[n]-e[n-1]\}+\frac{K_pT}{T_i}e[n]+\frac{K_pT_d}{T}\{e[n]-2e[n-1]+e[n-2]\}$$

令

$$K_i = K_p \frac{T}{T_i}$$

为积分系数；

$$K_d = K_p \frac{T_d}{T}$$

为微分系数，可以将上式简化为

$$\Delta u[n] = K_p \{e[n] - e[n-1]\} + K_i e[n] + K_d \{e[n] - 2e[n-1] + e[n-2]\}$$

增量式 PID 控制

增量式 PID 控制主要是通过求出增量，将原先的积分环节的累积作用进行了替换，避免积分环节占用大量计算性能和存储空间。

增量式 PID 控制的主要优点为：

①算式中不需要累加。控制增量Δu(k)的确定仅与最近 3 次的采样值有关，容易通过加权处理获得比较好的控制效果；

②计算机每次只输出控制增量，即对应执行机构位置的变化量，故机器发生故障时影响范围小、不会严重影响生产过程；

③手动—自动切换时冲击小。当控制从手动向自动切换时，可以作到无扰动切换 。

由于增量式需要对控制量进行记忆，所以对于不带记忆装置的系统，只能使用位置式 PID 控制方式进行控制。

4.5 amcl 参数的设置

4.6 move_base 主要参数的调整

根据查找以及对钢铁侠官方包的研究

了解了以下 param 设置

DWAPlannerROS:

acc_lim_th: 0.1 #旋转加速度极限

acc_lim_x: 0.5

acc_lim_y: 0.0 #y 方向的加速度绝对值,该值只有全向移动的机器人才需配置

max_vel_x: 0.5
min_vel_x: 0.0

max_vel_y: 0.0
min_vel_y: 0.0

max_trans_vel: 0.5 #robot 平移速度绝对值

min_trans_vel: 0.1

max_rot_vel: 0.2 #robot 旋转速度绝对值 defalt:1.0-0.4

min_rot_vel: 0.0

Trajectory Scoring Parameters
goal_distance_bias: 30.0
path_distance_bias: 20.0

occdist_scale: 0.01 #定义控制器躲避障碍物的程度

forward_point_distance: 0.5
stop_time_buffer: 0.2
scaling_speed: 0.5
max_scaling_factor: 0.2
##publish_cost_grid: false

Oscillation Prevention Parameters

oscillation_reset_dist: 0.05 #在振荡标志复位之前机器人必须以米为单位行进多远

Forward Simulation Parameters

vx_samples: 20 #探索 x 速度空间时要使用的样本数 defalt:3 vy 10 vth 20

vy_samples: 0
vtheta_samples: 10

sim_time: 1.5 #向前仿真轨迹的时间

sim_granularity: 0.025 #步长,轨迹上采样点之间的距离,轨迹上点的密集程度

rot_stopped_vel: 0.01
trans_stopped_vel: 0.01

Goal Tolerance Parameters
xy_goal_tolerance: 0.2
yaw_goal_tolerance: 0.1 #0.1 rad=5.7 degree

latch_xy_goal_tolerance: true #如果目标误差被锁定，若机器人达到目标 XY 位置，它

将旋转到位，即使误差没有达到，也会做旋转

shutdown_costmaps: false

controller_frequency: 10.0
controller_patience: 3.0

planner_frequency: 10.0 #全局路径规划器循环速率

planner_patience: 5.0

oscillation_timeout: 10.0 #恢复操作执行前允许震荡时间

oscillation_distance: 0.2 #必须移动多远被认为不震荡

clearing_rotation_allowed: false #此参数仅在使用默认恢复行为时使用，这意味着用

户尚未自定义 recovery_behaviors 参数

static_map:这个参数需要设置为 ture 否则将会不能按照雷达采集地图进行全局规划

inflation_radius: 同时这个参数决定着障碍物和全局的膨胀系数需要进行调试参考设置